

DAFFODIL INTERNATIONAL UNIVERSITY
FACULTY OF SCIENCE AND INFORMATION TECHNOLOGY
DEPARTMENT OF SOFTWARE ENGINEERING
Course: SE-231 (Software System Analysis & Design / Capstone Project 2)

CAPSTONE PROJECT PROPOSAL
PART 1: PROJECT PLANNING AND DEFINITION

RetailSync: Centralized Super Shop Warehouse Management System

*Real-Time POS Synchronization, Dynamic Putaway, FEFO Batch Tracking, and Algorithmic
Replenishment*

PROJECT SUBMISSION CREDENTIALS

Student / Author: Raisul Islam Likhon

Student ID / Section: SWE-44D

Degree Program: Bachelor of Science in Software Engineering (B.Sc. SWE)

Department: Department of Software Engineering, Daffodil International University

Academic Supervisor: Course Instructor & Evaluation Panel, Department of Software Engineering

Submission Term: Fall 2026 | Date: September 2026 | Version: 1.0.0-RELEASE

Executive Summary & Project Abstract

RetailSync is a high-performance, centralized enterprise Warehouse Management System (WMS) engineered specifically for the fast-evolving modern grocery retail and super shop sector in Bangladesh (e.g., Shwapno, Agora, Meena Bazar, Unimart, Daily Shopping). The platform bridges the destructive operational chasm between central distribution centers and frontline retail checkout counters (POS). By enforcing strict Third Normal Form (3NF) relational database integrity, sub-2.0s atomic POS inventory deductions, dynamic spatial putaway, and automated First-Expired, First-Out (FEFO) batch allocation, RetailSync directly eliminates the three largest profit leaks in modern trade: (1) perishable food spoilage (15% to 22% annual loss), (2) phantom inventory shrinkage (1.8% to 2.4% inventory write-offs), and (3) peak-hour stockouts during festival surges (7.5% to 11.2% lost revenue).

Architecturally, RetailSync is deployed as a 4-tier cyber-physical platform combining a Next.js 14 Progressive Web Application (PWA), an asynchronous FastAPI (Python 3.11+) backend, a normalized PostgreSQL 16 relational ledger, and an in-memory Redis 7 caching and idempotency tier. To minimize capital expenditure, RetailSync adopts a frugal hardware strategy pairing standard consumer Android smartphones with low-cost Bluetooth barcode trigger grips (< 4,000 BDT), eliminating the need for expensive 60,000 BDT industrial handheld terminals while maintaining native compatibility with stationary automated dock gate scanners (ESP32 MCU over MQTT/TLS).

CORE CAPSTONE THESIS & QUANTIFIABLE SLO TARGETS

Primary Research Question: How can centralized relational concurrency and automated FEFO prioritization eliminate retail stockouts and perishable spoilage in high-density grocery operations?

Key SLO Target: Sub-2.0s POS checkout deduction latency across 10 concurrent branch registers with 0 deadlocks.

Methodology: 14-Week Agile Scrum Lifecycle delivering an incremental, Dockerized production-grade MVP across 6 sprints.

Document Architecture & Table of Contents

This project proposal is structured into ten cohesive chapters providing comprehensive planning, empirical justification, and architectural design for RetailSync in accordance with DIU Capstone standards:

Section	Title	Summary Scope
Section 1	Industry Background & Problem Statement	Operational context, profit leaks, and problem-to-feature mapping
Section 2	Project Objectives & Scope Boundaries	SMART goals, MoSCoW prioritization matrix, in-scope vs out-of-scope
Section 3	Target Personas & Operational Workflows	User profiles (Manager, Cashier, Operator, Procurement) and use cases
Section 4	Core Functional Modules & Feature Breakdown	8 core modules: Auth, POs, Inbound, Putaway, FEFO, POS, DSS, Audit
Section 5	Non-Functional Requirements & Performance SLOs	Latency, throughput, ACID concurrency, security, and food safety compliance
Section 6	System Architecture & Technical Design	4-tier architecture, PostgreSQL 16 schema, row-level locking, offline sync
Section 7	Curated Technology Stack & Hardware Strategy	Next.js 14 PWA, FastAPI, PostgreSQL, Redis, and frugal barcode scanner model
Section 8	Development Methodology: Agile Scrum Framework	Scrum rationale, 14-week sprint roadmap, sprint backlogs, Definition of Done
Section 9	Resource Allocation, Budget & Risk Management	RACI matrix, hardware expenditure (< 35,000 BDT), and local risk mitigations
Section 10	Verification, Testing, ROI Impact & Conclusion	Testing strategy, expected financial ROI, academic roadmap, and references

1. Industry Background & Problem Statement

1.1 The Modern Grocery Retail Landscape in Bangladesh

The supermarket and organized grocery retail sector in Bangladesh is experiencing rapid compound annual growth driven by rising urbanization, dual-income households, and an expanding middle class seeking convenience and food safety assurances. Major retail chains such as Shwapno (over 450 outlets), Agora, Meena Bazar, Unimart, and Daily Shopping handle thousands of high-velocity fast-moving consumer goods (FMCG) and perishable food items daily across dense metropolitan hubs like Dhaka and Chattogram.

However, behind modern retail checkout counters lies a fragile, disjointed warehouse and supply chain infrastructure. While front-of-house Point of Sale (POS) billing systems have modernized, back-of-house warehouse storage, stock allocation, and central distribution center operations remain heavily dependent on manual pen-and-paper logs, fragmented spreadsheets, or legacy ERP modules lacking real-time synchronization.

1.2 The Three Critical Profit Leaks in Super Shop Operations

- **1. Perishable Food & Dairy Spoilage:** Modern supermarkets lose 15% to 22% of perishable inventory annually due to lack of batch-level expiration visibility. Floor workers routinely pull newer stock from front shelves while older batches rot unnoticed at the back of warehouse bins, directly violating the Bangladesh Food Safety Act 2013.
- **2. Phantom Inventory & Operational Shrinkage:** A severe operational disconnect exists between what the central database reports and what physically exists on warehouse shelves. This 'phantom inventory' (1.8% to 2.4% shrinkage write-offs) stems from untracked damage, unrecorded pilferage, and delayed end-of-day spreadsheet reconciliations.
- **3. Peak-Hour Stockouts & POS Freezing:** During peak shopping windows (Ramadan, Eid-ul-Fitr, weekend evenings), high-velocity items (e.g., 1L soyabean oil, milk, sugar) sell out in minutes. Because POS registers do not deduct warehouse stock atomically, replenishment orders lag by 6 to 12 hours, resulting in 7.5% to 11.2% lost revenue from customer walkouts.

1.3 Operational Gaps in Current Commercial Solutions

Existing commercial Enterprise Resource Planning (ERP) systems and legacy WMS platforms fail Bangladeshi super shops across three critical dimensions: (1) Prohibitive Cost: Enterprise solutions (SAP, Oracle, Odoo Enterprise) demand tens of thousands of dollars in licensing and require 60,000 BDT industrial handheld terminals; (2) Batch Blindness: Generic ERPs treat stock as aggregate quantities without enforcing strict First-Expired, First-Out (FEFO) picking queues; and (3) Network Fragility: Cloud-only systems freeze completely during frequent local broadband outages, halting checkout queues.

1.4 Problem-to-Feature Mapping

ID	Real-World Problem	Operational Impact	RetailSync Solution
P-01	Data Isolation	Sales data lives in the POS; warehouse data lives on clipboards.	Centralized SQL Database serving both POS and Warehouse APIs.
P-02	Undetected Stockouts	High-demand items run out on shelves before restock is ordered.	Automated Reorder Point (ROP) triggers sending alerts to managers.

ID	Real-World Problem	Operational Impact	RetailSync Solution
P-03	Perishable Waste	Lack of batch tracking leads to expired goods.	Strict FIFO/FEFO logic and expiry-date dashboard highlights.
P-04	Supplier Delays	Manual PO generation is slow and error-prone.	Automated PO generation based on low-stock metrics.
P-05	Internal Shrinkage	Untraceable manual stock adjustments.	Role-Based Access Control (RBAC) with immutable audit logging.

2. Project Objectives & Scope Boundaries

2.1 SMART Project Objectives

Objective Area	SMART Target & Engineering Metric
Sub-2.0s POS Concurrency	Execute atomic stock deductions from retail cash registers via PostgreSQL row-level locks, achieving p95 latency ≤ 800ms with zero overselling across 10 concurrent registers.
Automated FEFO Priority	Enforce strict First-Expired, First-Out picking logic that mechanically quarantines batches within 3 days of expiry, reducing perishable spoilage from 22% to under 6%.
Algorithmic Replenishment DSS	Integrate Dynamic Economic Order Quantity (EOQ) and Greasley's Statistical Safety Stock factoring vendor lead-time variances, reducing stockouts by > 60%.
Frugal Hardware Architecture	Engineer a responsive Progressive Web Application (PWA) compatible with consumer Android smartphones and Bluetooth trigger scanners (< 4,000 BDT), cutting hardware costs by 90%.
Offline Network Resilience	Implement client-side Service Workers and encrypted IndexedDB queues to buffer up to 500 sales transactions during internet dropouts, replaying idempotently upon recovery.

2.2 Scope Boundaries & MoSCoW Prioritization

To guarantee successful completion and rigorous validation within the academic capstone timeframe, the project boundaries are structured into four MoSCoW categories:

Category	Feature Scope	Timeline	Deliverable
Category	In-Scope Modules / Engineering Features	Sprint Alignment	Target Deliverable

Category	Feature Scope	Timeline	Deliverable
Must Have (MVP)	Auth & RBAC, Master Catalog, Supplier & Digital POs, Inbound Barcode Scanning & GRN, Spatial Putaway, Real-Time FEFO Ledger, Sub-2.0s POS Deductions, Dynamic EOQ & Safety Stock DSS.	Sprints 1 to 4	Functional core platform operating end-to-end.
Should Have (Enhancements)	Outbound Wave Picking, Branch Store Requisition Portal, Blind Cycle Counting with Discrepancy Workflows, Offline POS Service Worker Caching with Idempotent Auto-Sync.	Sprints 5 to 6	Hardened enterprise feature set for multi-branch trials.
Could Have (Future Post-Capstone)	Unsupervised ML Shrinkage Anomaly Detection (Isolation Forest), Stationary Automated Dock Gate Scanner (ESP32 MCU over MQTT/TLS), BLE Cold-Chain Sensor Telemetry.	Future Roadmap	Proof-of-concept enterprise edge expansions.
Won't Have (Out of Scope)	Autonomous robotic picking cranes (AGVs), physical warehouse conveyor belts, full corporate double-entry general ledger accounting ERP, custom silicon hardware.	Out of Scope	Avoids unrealistic hardware and accounting bloat.

3. Target Personas & Core Use-Case Workflows

3.1 Core Target Personas

RetailSync is designed around the daily operational realities of four primary user personas in modern trade:

User Persona	Role Description	System Needs
Store Manager	Oversees supply chain and profitability.	Needs automated PO drafts, shrinkage reports, and a macro-view of inventory health.
Cashier	Processes direct consumer sales.	Requires a minimalist interface. The system must never freeze during a scan, even if the internet drops momentarily.
Inventory Clerk	Handles loading docks and physical counting.	Needs fast, barcode-compatible interfaces for receiving large shipments accurately.

3.2 End-to-End Operational Lifecycle Workflow

The end-to-end operational lifecycle connects physical goods movement to atomic database state transitions:

Step 1: Inbound Receiving & GRN Verification: When a supplier truck arrives at the warehouse dock, the receiving clerk interrogates purchase order line items by scanning carton barcodes. The system captures manufacturer batch numbers, production dates, and expiration dates, generating a digital Goods Receipt Note (GRN) with automatic variance tagging.

Step 2: Directed Spatial Putaway: Upon GRN confirmation, the system calculates the optimal storage coordinate (Zone-Aisle-Rack-Shelf-Bin) based on product category, temperature zone (Chilled, Ambient, Frozen), and SKU turnover velocity. The operator confirms docking by scanning the physical bin barcode.

Step 3: Real-Time POS Inventory Deduction: When a retail customer purchases an item at the frontline checkout register, the POS sends an atomic deduction request to the API. The system executes a pessimistic row lock ('SELECT ... FOR UPDATE') on the earliest active batch, deducting stock in < 2.0 seconds with zero overselling.

Step 4: Strict FEFO Outbound Allocation: When supermarket branches submit store replenishment requisitions, the picking engine allocates stock strictly from the earliest expiring available batch. Expired or quarantined lots are mechanically excluded from pick lists.

Step 5: Algorithmic Replenishment Trigger: Every stock transaction updates the net available balance. When stock breaches the calculated Reorder Point (ROP), the system triggers an automated replenishment advisory populating a draft PO with the mathematically optimal EOQ quantity.

4. Core Functional Modules & Feature Breakdown

RetailSync decomposes complex warehouse operations into eight cohesive, modular domain services:

M-01: Authentication, RBAC & Audit Ledger: Manages cryptographic Argon2id password hashing, stateless JWT session tokens (15-min access, 8-hour refresh), and role-based permissions isolating Cashier, Floor Operator, Clerk, Supervisor, Procurement, and Admin roles. Every stock transition appends to an immutable audit ledger.

M-02: Master Catalog & Supplier PO Lifecycle: Maintains product SKUs, EAN-13 barcodes, storage temperature classifications, and supplier master records. Orchestrates digital Purchase Order workflows through DRAFT, ISSUED, PARTIAL, COMPLETED, and CANCELLED states.

M-03: Handheld Inbound Receiving & Digital GRN: Enables mobile camera and Bluetooth barcode interrogation of incoming deliveries against active POs. Captures batch numbers and expiration dates, calculates short-shipment variances, and generates verified digital GRNs.

M-04: Directed Spatial Putaway Engine: Calculates optimal 3D bin coordinates factoring SKU velocity (Class-A items docked near dispatch doors), climate zones (Chiller, Deep Freezer, Ambient), and shelf weight capacities, requiring two-scan confirmation.

M-05: Real-Time FEFO Inventory Ledger: Maintains batch-level granularity (`product\_batches`). Automates First-Expired, First-Out picking allocation and executes nightly background triggers to transition expiring batches to QUARANTINED and EXPIRED states.

M-06: Sub-2.0s Point of Sale (POS) Concurrency Sync: Exposes high-performance transactional REST deduction endpoints. Implements pessimistic row-level locking (`SELECT ... FOR UPDATE`) to guarantee atomic stock updates with zero deadlocks and zero phantom sales.

M-07: Algorithmic Replenishment DSS: Integrates statistical algorithms to eliminate stockouts without excessive holding costs. Computes Dynamic Economic Order Quantity (EOQ), Greasley's Statistical Safety Stock, and dynamic Reorder Points (ROP) as formulated below:

M-08: Continuous Blind Cycle Counting: Generates daily ABC-classified cycle counting task sheets where expected system quantities are hidden from floor workers to prevent confirmation bias, routing discrepancies above 2% for supervisory review.

MATHEMATICAL FORMULATIONS FOR REPLENISHMENT DSS (MODULE M-07)

1. Dynamic Economic Order Quantity: $EOQ = \sqrt{(2 \times D \times S) / H}$ [D = Annual Demand, S = Setup/Order Cost, H = Holding Cost per unit/year]

2. Greasley's Statistical Safety Stock: $SS = Z \times \sqrt{(\bar{L} \times \sigma_d^2) + (\bar{d}^2 \times \sigma_L^2)}$ [Factoring demand variance σ_d^2 AND lead-time variance σ_L^2]

3. Dynamic Reorder Point: $ROP = (\bar{d} \times \bar{L}) + SS$ [Triggering automated replenishment advisories when net stock $\leq ROP$]

4.9 Core Functional Requirements Summary

ID	Module	Specific Requirement	Priority
FR-01	Auth	System shall enforce RBAC isolating Cashier, Clerk, and Manager views.	Must
FR-10	Inventory	System shall track Product ID, SKU, Batch Number, and Expiry Date.	Must

ID	Module	Specific Requirement	Priority
FR-11	Inventory	System shall allow status tagging: Available, Quarantined, Sold.	Must
FR-20	Receiving	System shall generate a Goods Receipt Note (GRN) linked to a specific PO.	Must
FR-21	Receiving	Clerks can flag received items as 'Damaged' upon intake.	Should
FR-30	POS Sync	API shall deduct inventory based on strict FIFO/FEFO batch logic.	Must
FR-40	Alerts	System shall allow Managers to configure a Reorder Point (ROP) per SKU.	Must
FR-41	Alerts	System shall flag batches expiring within X days (configurable).	Must
FR-50	Auditing	Every stock modification must log the timestamp, User ID, and old/new value.	Must

5. Non-Functional Requirements & Performance SLOs

To ensure production readiness in high-velocity retail environments, RetailSync enforces strict, measurable Non-Functional Requirements (NFRs) verified through automated benchmarking:

ID	Category	Specific Metric
NFR-01	Latency	The POS deduction API must respond in under 2.0 seconds under normal load.
NFR-02	Data Integrity	Database must use transaction blocks to prevent partial updates if a crash occurs.
NFR-03	Security	All passwords must be hashed (bcrypt/Argon2id). No plaintext credentials stored.
NFR-04	Concurrency	System must handle at least 5 simultaneous POS registers deducting stock.
NFR-05	Auditability	Audit logs must be immutable (append-only) via database-level restrictions.

5.2 Food Safety & Regulatory Compliance Framework

RetailSync is engineered to guarantee strict compliance with the Bangladesh Food Safety Act 2013 and BSTI consumer protection mandates. The system enforces an automated ****Quarantine Lock****: any batch reaching within 3 days of expiration or flagged during receiving as defective is mechanically excluded from retail pick lists and POS barcode scanners, completely eliminating the legal and brand liability of selling expired food items.

6. System Architecture & Technical Design

6.1 4-Tier Cyber-Physical Architecture

RetailSync follows a decoupled, 4-tier cyber-physical architecture designed for resilience and sub-second latency:

Tier 1: Client Edge (Handheld & POS PWA): Responsive Progressive Web Application built in Next.js 14 and Tailwind CSS. Operates on warehouse floor smartphones, tablets, and POS desktop terminals. Integrates pure JavaScript barcode engines (ZXing / Html5-QRCode) and Service Worker IndexedDB offline queues.

Tier 2: Edge Gateway & Security Layer: Reverse proxy (Nginx) terminating TLS 1.3 encryption, managing rate limiting, CORS policies, and local edge routing. For stationary dock gates, an ESP32 microcontroller with RFID/fixed barcode readers communicates over MQTT with TLS.

Tier 3: Core Application Services Tier: Stateless, asynchronous REST API powered by Python 3.11+ and FastAPI. Contains isolated domain modules for Inbound, Putaway, FEFO Picking, Replenishment DSS, and Audit Logging, containerized via Docker.

Tier 4: Enterprise Data & In-Memory Storage: PostgreSQL 16 relational database with strict 3NF normalization, foreign key referential integrity, and composite B-Tree indexes. Paired with Redis 7 for in-memory session management, idempotency key caching, and distributed rate locks.

6.2 High-Throughput Concurrency Control: Row-Level Locking

The central engineering challenge during supermarket rush hours is preventing overselling when multiple cash registers ring up the same high-velocity SKU simultaneously. RetailSync solves this via pessimistic row-level locking in PostgreSQL:

When an `/api/v1/pos/sync`` deduction transaction begins, the database queries `product_batches`` using `SELECT id, current_qty FROM product_batches WHERE product_id = :p_id AND current_qty > 0 ORDER BY expiry_date ASC LIMIT 1 FOR UPDATE``. This locks exclusively that specific batch record. Subsequent concurrent POS requests for the same batch queue safely for milliseconds without deadlocking or reading stale stock balances. Once the deduction is written to the ledger, the lock releases, guaranteeing absolute ACID consistency.

6.3 Relational Database Schema Overview (15 Tables)

Table Name	Primary Purpose & Operational Entity	Primary Key & Unique Keys	Key Foreign Key Constraints	Index Optimization Strategy
suppliers	Supplier directory, contact terms, lead-time variance metrics for Greasley safety stock.	supplier_id (PK)	None (Root entity)	B-Tree on company_name, email
products	Master SKU catalog, category classification, storage temperature, unit & holding costs.	product_id (PK), sku_code (UQ), barcode_ean (UQ)	supplier_id -> suppliers(supplier_id)	B-Tree on sku_code, barcode_ean, category
locations	Warehouse spatial coordinates: Zone, Aisle, Rack, Shelf Tier, Bin, and capacity limits.	location_id (PK), bin_barcode (UQ)	None (Spatial domain entity)	B-Tree on bin_barcode, zone_code
product_batches	Perishable batch lots, manufacturing date, expiration date, on-hand qty, quarantine status.	batch_id (PK), batch_lot_number (UQ)	product_id -> products(product_id), location_id -> locations(location_id)	B-Tree on expiry_date, product_id, status (FEFO index)
purchase_orders	Digital PO tracking, supplier link, approval lifecycle, total cost, delivery deadline.	po_id (PK), po_number (UQ)	supplier_id -> suppliers(supplier_id)	B-Tree on po_number, status, order_date

Table Name	Primary Purpose & Operational Entity	Primary Key & Unique Keys	Key Foreign Key Constraints	Index Optimization Strategy
goods_receipt_notes	Inbound dock inspection, PO line validation, accepted vs damaged count, digital invoice link.	grn_id (PK), grn_number (UQ)	po_id -> purchase_orders(po_id)	B-Tree on grn_number, po_id, received_at
inventory_transactions	Immutable append-only ACID ledger recording all stock movements with row-locking support.	transaction_id (PK, BIGSERIAL)	product_id, batch_id, location_id, user_id	Composite B-Tree on (product_id, recorded_at DESC), batch_id
pos_registers	Authorized branch super shop checkout cash registers and real-time sync telemetry.	register_id (PK), terminal_mac_ip (UQ)	branch_store_id (Reference)	B-Tree on register_code, store_code, status

7. Curated Technology Stack & Hardware Strategy

7.1 Curated Technology Stack Matrix

Component	Recommended Technology	Justification for Capstone
Database Layer	MySQL / PostgreSQL	Relational architecture is mandatory for retail to ensure ACID compliance (Atomicity, Consistency, Isolation, Durability) for financial transactions.
Backend API	Python (Django/FastAPI) or Java	Excellent handling of concurrent requests, complex rule engines, and automated task scheduling (e.g., nightly expiry checks).
Frontend Dashboard	React / Next.js	Provides a highly responsive, single-page application (SPA) experience for managers, eliminating page-reload delays.
Local Testing Env.	XAMPP / Beekeeper Studio	Standard, accessible environment for developing and visualizing the relational schema during the capstone phase.

7.2 Architectural Decision Records (ADRs)

ADR-01: Next.js PWA over Native Android App: Building as a Progressive Web Application eliminates app store review friction and device MDM management. Any update deployed to the web server is instantly available to all floor devices upon refresh. Service Workers and IndexedDB provide native-grade offline caching.

ADR-02: FastAPI Asynchronous ASGI over Django/Flask: FastAPI's native Python async/await event loop provides high-concurrency I/O performance capable of handling thousands of requests per second, with automatic OpenAPI 3.1 documentation and Pydantic v2 data validation.

ADR-03: Normalized PostgreSQL 16 over MongoDB / NoSQL: Inventory transactions represent legal and financial records requiring strict ACID double-entry ledger guarantees. PostgreSQL foreign keys and row locks prevent orphaned transactions and phantom inventory anomalies.

ADR-04: Redis 7 In-Memory Caching & Idempotency: Redis provides sub-millisecond validation of `X-Idempotency-Key` headers on POS sync endpoints, preventing duplicate sales deductions during network retries while caching active user sessions.

7.3 Frugal Barcode Hardware Strategy

Traditional enterprise WMS implementations fail in developing economies due to exorbitant hardware costs: a single ruggedized industrial terminal (e.g., Zebra TC52) costs upwards of 60,000 BDT. RetailSync disrupts this paradigm by adopting a ****frugal hardware model****: the web application runs on standard consumer Android smartphones (10,000 to 12,000 BDT) paired via Bluetooth HID with ergonomic wireless barcode trigger grips (< 4,000 BDT) or utilizing the smartphone's camera via `Html5-QRCode`. This slashes terminal acquisition costs by over 75% while maintaining rapid sub-350ms barcode scan speeds.

8. Development Methodology: Agile Scrum Framework

8.1 Methodology Rationale & Governance

RetailSync is developed strictly following the **Agile Scrum Framework**. We deliberately bypass rigid sequential models (such as Waterfall or V-Model) because modern retail systems require rapid empirical validation, iterative stakeholder feedback from super shop managers, and continuous refinement of concurrency and picking algorithms.

- **Sprint Cadence:** Development is divided into six 2-week iterations followed by a 2-week hardening and defense phase (14 weeks total).
- **Sprint Planning:** 2-hour session on Day 1 of each sprint to commit user stories from the prioritized product backlog.
- **Daily Standup:** Daily 10-minute standup tracking completed work, planned tasks, and technical blockers.
- **Sprint Review & Demo:** Bi-weekly live software demonstration to academic supervisors and evaluation panel.

8.2 14-Week Capstone Implementation Roadmap

Academic Timeframe	Core Engineering Modules & Deliverables	Key Technical Milestones
Weeks 1-3	Database Normalization (MySQL/PostgreSQL), ERD design, and API scaffolding (Python/Java).	Relational schema definition, 3NF normalization, baseline REST endpoints.
Weeks 4-6	Inventory CRUD, Batch Management, and Expiry Logic implementation.	Batch tracking, dynamic shelf-life calculations, FEFO/FIFO queue logic.
Weeks 7-9	Supplier module, PO generation, and GRN workflows.	Supplier SLA records, digital PO lifecycle, dock inspection & GRN generation.
Weeks 10-11	POS Sync Simulation (building the checkout deduction logic and FIFO handling).	Real-time POS API, atomic stock deduction, concurrency locking (<2s latency).
Weeks 12-13	Rule engines for low-stock alerts, reporting dashboards, and audit logs.	Automated ROP triggers, draft PO generation, immutable audit logging.
Week 14	System testing, concurrency stress testing, and final defense preparation.	Stress testing (5+ concurrent POS), UAT validation, capstone presentation & defense.

8.3 Definition of Done (DoD) & Quality Gates

A user story or sprint task is declared complete only when it satisfies the following non-negotiable Definition of Done:

1. Code Complete: Feature implemented, commented, and committed to Git with conventional commit messages.
2. Unit & Integration Tests Passing: Automated test suite passes with > 80% line coverage.
3. Concurrency Validated: Multi-threaded race condition tests confirm zero deadlocks and zero negative stock balances.

4. API Documented: Endpoint registered in FastAPI with accurate Pydantic request/response schemas in OpenAPI.
5. Dockerized: Changes build and run cleanly via `docker compose up -d` without manual host modifications.
6. Mobile Responsive: Handheld UI verified on mobile viewport (375x667) with touch-friendly hit targets.

9. Resource Allocation, Budget & Risk Management

9.1 Team Roles & RACI Matrix

To ensure accountability across engineering modules, project tasks are mapped via a formal RACI Matrix (Responsible, Accountable, Consulted, Informed):

Project Lifecycle Phase / Core Deliverable	Floor Staff / Cashiers	Floor Supervisors	Procurement Officers	Store Managers	DevOps / SysAdmin	Developer Team	Academic Committee
Requirements Elicitation & Domain Study	Consulted	Consulted	Consulted	Consulted	Informed	Responsible / Accountable	Informed / Approver
Relational 3NF Database Schema & DDL	Informed	Consulted	Informed	Informed	Consulted	Responsible / Accountable	Consulted
ACID Concurrency & Row-Locking Engine	Informed	Informed	Informed	Informed	Consulted	Responsible / Accountable	Informed
Sub-2.0s POS Sync & Real-Time API	Consulted	Informed	Informed	Consulted	Consulted	Responsible / Accountable	Informed
FEFO/FIFO Dynamic Queue Algorithm	Informed	Consulted	Consulted	Consulted	Informed	Responsible / Accountable	Consulted
Greasley Safety Stock & EOQ Engine	Informed	Consulted	Responsible	Consulted	Informed	Accountable	Informed
Mobile Barcode Scanning PWA Interface	Responsible	Responsible	Informed	Informed	Informed	Accountable	Informed

Project Lifecycle Phase / Core Deliverable	Floor Staff / Cashiers	Floor Supervisors	Procurement Officers	Store Managers	DevOps / SysAdmin	Developer Team	Academic Committee
Immutable Audit Ledger & Security	Informed	Consulted	Consulted	Informed	Responsible	Accountable	Informed
Dual-Level Black-Box & White-Box QA	Consulted	Consulted	Consulted	Consulted	Consulted	Responsible / Accountable	Consulted / Evaluator
Final Manuscript, System Demo & Defense	Informed	Informed	Informed	Informed	Informed	Responsible / Accountable	Accountable / Approver

9.2 Hardware & Operational Budget Breakdown

Hardware / Component	Specification / Unit Model	Qty	Unit Cost (BDT)	Total Cost (BDT)
Android Test Smartphone	Redmi / Realme 6.5" Android 13, 4GB RAM	1	12,500	12,500
Bluetooth Barcode Trigger Grip	Netum / Eyoyo 1D/2D Bluetooth HID Scanner	2	3,800	7,600
Thermal Label / Receipt Printer	Xprinter 4-inch USB/Bluetooth Thermal Printer	1	6,500	6,500
EAN-128 Warehouse Barcode Labels	Roll of 1,000 thermal adhesive labels (4x6 in)	3	650	1,950
Cloud Staging VPS Hosting	DigitalOcean / Hetzner 2 vCPU, 4GB RAM (6 Mos)	1	5,500	5,500
Total Estimated Budget	Complete Capstone Hardware & Staging Prototype	—	—	34,050 BDT

9.3 Local Operational Risk Assessment & Proactive Mitigations

ID	Technical Risk	Likelihood	Context & Mitigation Strategy
R-01	Database Concurrency Conflicts	High	Two cashiers sell the last unit of milk at the same millisecond. Mitigation: Use SQL Row-Level Locking (SELECT ... FOR UPDATE).
R-02	Internet Outages	High	Store loses connection to the central server. Mitigation: Implement a queue system on the POS that caches sales and bulk-syncs when online.
R-03	Data Entry Errors at Receiving	Medium	Clerk enters 500 instead of 50 boxes. Mitigation: System warns if GRN exceeds the Purchase Order quantity by >5%.

10. Verification, Validation, Expected Impact & Conclusion

10.1 Multi-Tier Verification & Testing Strategy

Testing Tier & Category	Target Operational Dimension	Test Case Scenario & Verification Methodology	Verifiable Pass / Acceptance Criteria
Tier 1: Black-Box Functional	Barcode Interrogation & Scanning Latency	Interrogate 1D/2D barcodes on mobile cameras and Bluetooth triggers across low and high ambient lighting (100–600 lux).	100% SKU identification accuracy; UI updates and audible/haptic feedback trigger within <= 350 ms.
Tier 1: Black-Box Functional	Inbound GRN vs. PO Automated Variance	Simulate delivery of 50 cartons with 3 cartons flagged as crushed; generate digital GRN against open PO.	GRN logs 47 accepted, 3 quarantined; automated supplier credit note generated; zero quarantined stock added to POS.
Tier 1: Black-Box Functional	Offline Network Resilience & Bulk Sync	Sever store ISP connection during 25 consecutive barcode checkouts; reconnect network after 5 minutes.	Local client queue caches all 25 sales with timestamps; bulk POST executes automatically on reconnect with zero dropped sales.

Testing Tier & Category	Target Operational Dimension	Test Case Scenario & Verification Methodology	Verifiable Pass / Acceptance Criteria
Tier 2: White-Box Structural	ACID Concurrency & Row-Locking ('SELECT FOR UPDATE')	Launch 10 parallel asynchronous POS worker threads attempting to purchase the last 2 available units of a dairy SKU simultaneously.	Exactly 2 threads succeed; 8 threads receive clean out-of-stock messages; inventory balance remains exactly 0; zero database deadlocks.
Tier 2: White-Box Structural	Strict FEFO Priority Queue Verification	Seed database with 3 batches of milk expiring in 5 days, 15 days, and 45 days; issue automated checkout sales deductions.	Database triggers strictly decrement stock from the 5-day expiry batch first; automatically switches to 15-day batch only when 5-day is depleted.
Tier 2: White-Box Structural	SQL Injection & XSS Penetration Defense	Inject SQL injection strings ('OR 1=1; DROP TABLE) and XSS payloads into barcode search, user login, and PO creation fields.	All inputs sanitized via parameterized queries and ORM; malicious payloads rejected with HTTP 400; security access log records attack attempt.
Tier 2: White-Box Structural	Mathematical Replenishment Engine Precision	Execute algorithmic unit test suite across 100 historical SKU sales and lead-time distributions (EOQ, Greasley, ROP).	Computed EOQ, Safety Stock, and ROP match analytical statistical benchmarks within 0.01% floating-point tolerance.

10.2 Expected Operational & Financial Impact (ROI)

Performance Metric	Traditional Super Shop Baseline	RetailSync Target Value	Operational Benefit
Perishable Food Spoilage	15% to 22% annual dairy/produce loss	< 6% annual loss	Saves millions of BDT in prevented food waste write-offs.
POS Checkout Scan Latency	3.0 to 6.0s (or database freezes)	≤ 2.0s (p95 ≤ 800ms)	Eliminates customer checkout queue abandonment.
Phantom Inventory Shrinkage	1.8% to 2.4% unexplained loss	< 0.4% discrepancy	Complete traceability via immutable audit ledger.
Inbound Dock Unloading Time	45 to 75 minutes per truck	< 15 minutes per truck	Instant barcode interrogation against digital PO line items.

Performance Metric	Traditional Super Shop Baseline	RetailSync Target Value	Operational Benefit
Stockout Frequency (Rush Hours)	7.5% to 11.2% lost sales	< 2.5% stockout rate	Automated replenishment alerts based on statistical safety stock.

10.3 Capstone Part 1 Conclusion & Transition to Part 2

This project proposal establishes the foundational planning, empirical justification, and architectural blueprint for RetailSync. By addressing the critical structural weaknesses of the Bangladeshi modern trade sector—namely perishable food waste, unrecorded shrinkage, and peak-hour stockouts—RetailSync provides a frugal, scalable, and technically rigorous software solution. The completion of Part 1 (Project Planning and Definition) positions the project for immediate, structured execution in Part 2 (System Design, Prototyping, and Concurrency Validation) according to the 14-week Agile Scrum roadmap.

10.4 Key Academic & Industry References

1. Bangladesh Food Safety Authority (BFSA), 'Food Safety Act 2013,' Ministry of Food, Government of the People's Republic of Bangladesh, Dhaka, 2013.
2. Greasley, A., 'Operations Management,' 3rd ed., John Wiley & Sons, New York, 2013 (Statistical Safety Stock under dual variance models).
3. Silver, E. A., Pyke, D. F., and Peterson, R., 'Inventory Management and Production Planning and Scheduling,' 3rd ed., Wiley, 1998 (Dynamic EOQ algorithms).
4. Kleppmann, M., 'Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems,' O'Reilly Media, 2017.
5. PostgreSQL Global Development Group, 'PostgreSQL 16 Documentation: Concurrency Control and Explicit Locking,' 2024. [Online]. Available: <https://www.postgresql.org/docs/16/explicit-locking.html>
6. Schwaber, K. and Sutherland, J., 'The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game,' Scrum.org, 2020.
7. Bangladesh Bureau of Statistics (BBS), 'Report on Wholesale and Retail Trade Survey in Bangladesh,' Ministry of Planning, Dhaka, 2022.